Implementação de sistema cadastral com interface Web, baseado nas tecnologias de Servlets, JPA e JEE.

Nome do Campus: Polo Jd Santa Cecília – Sorocaba – SP

Nome do Curso: Desenvolvimento FullStack

Nome da Disciplina: DGT2811 Desenvolvimento Back-End Corporativo com Java e Cloud

Número da Turma: 9001

Semestre: 6° semestre

Nome do Integrante: Guilherme Costa Barbosa

Link do Repositório:
https://github.com/GuilhermeCostaBarbosa/SistemaCadastroProdutoJavaEstacio

Implementação de sistema cadastral com interface Web, baseado nas tecnologias de Servlets, JPA e JEE.

Objetivo da Prática:

- Implementar persistência com base em JPA.
- Implementar regras de negócio na plataforma JEE, através de EJBs
- Implementar sistema cadastral Web com base em Servlets e JSPs.
- Utilizar a biblioteca Bootstrap para melhoria do design.
- Criar os elementos necessários para exibição e entrada de dados na plataforma Java Web.

Códigos Solicitados

Código do ServletProdutoFC.java:

```
/*

 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license

 * Click
nbfs://nbhost/SystemFileSystem/Templates/JSP_Servlet/Servlet.java to
edit this template

 */

package cadastroee.servlets;


import cadastroee.controller.ProdutoFacadeLocal;

import cadastroee.model.Produto;

import jakarta.ejb.EJB;

import java.io.IOException;

import jakarta.servlet.ServletException;

import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class ServletProdutoFC extends HttpServlet {

    @EJB
    ProdutoFacadeLocal facade;

    protected void processRequest(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {

        // pegar parametro acao
        String acao = request.getParameter("acao");
        if (acao == null) {
            acao = "listar";
        }

        // Definir destino
        String destino = "ProdutoLista.jsp";

        // case acao
        switch (acao) {
            case "listar":
                request.setAttribute("produtos", facade.findAll());
                destino = "ProdutoLista.jsp";
                break;

            case "formIncluir":
                destino = "ProdutoDados.jsp";
                break;

            case "formAlterar":
```

```
                int idForm =
Integer.parseInt(request.getParameter("id"));
                request.setAttribute("produto", facade.find(idForm));
                destino = "ProdutoDados.jsp";
                break;

            case "incluir":
                Produto pNovo = new Produto();
                pNovo.setNome(request.getParameter("nome"));

pNovo.setQuantidade(Integer.parseInt(request.getParameter("quantidade"
)));

pNovo.setPrecoVenda(Float.parseFloat(request.getParameter("precoVenda"
)));
                facade.create(pNovo);

                request.setAttribute("produtos", facade.findAll());
                destino = "ProdutoLista.jsp";
                break;

            case "alterar":
                int idAlt =
Integer.parseInt(request.getParameter("id"));
                Produto pAlt = facade.find(idAlt);
                pAlt.setNome(request.getParameter("nome"));

pAlt.setQuantidade(Integer.parseInt(request.getParameter("quantidade")
));

pAlt.setPrecoVenda(Float.parseFloat(request.getParameter("precoVenda")
));
                facade.edit(pAlt);

                request.setAttribute("produtos", facade.findAll());
                destino = "ProdutoLista.jsp";
                break;
```

```
            case "excluir":
                int idExc =
Integer.parseInt(request.getParameter("id"));
                Produto pExc = facade.find(idExc);
                facade.remove(pExc);

                request.setAttribute("produtos", facade.findAll());
                destino = "ProdutoLista.jsp";
                break;
        }

        // Redirecionar para destino
        request.getRequestDispatcher(destino).forward(request,
response);
    }

    @Override
    protected void doGet(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    public String getServletInfo() {
        return "Front Controller para Produtos";
    }
```

```
}
```

Código do ProdutoLista.jsp:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<%@page import="java.util.List"%>

<%@page import="cadastroee.model.Produto"%>

<!DOCTYPE html>

<html>

    <head>

        <meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">

        <title>Listagem de Produtos</title>

        <!-- Link para o CSS do Bootstrap via CDN -->

        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">

    </head>

    <body class="container">

        <h1 class="mt-4">Listagem de Produtos</h1>


        <a href="ServletProdutoFC?acao=formIncluir" class="btn btn-
primary m-2">Novo Produto</a>

        <br><br>


        <table class="table table-striped">

            <thead class="table-dark">

                <tr>

                    <th>#</th>

                    <th>Nome</th>

                    <th>Quantidade</th>

                    <th>Preco de Venda</th>

                    <th>Opções</th>

                </tr>

            </thead>
```

```
            <tbody>
                <%
                    List<Produto> produtos = (List<Produto>)
request.getAttribute("produtos");
                    if (produtos != null) {
                        for (Produto p : produtos) {
                %>
                <tr>
                    <td><%= p.getIdProduto() %></td>
                    <td><%= p.getNome() %></td>
                    <td><%= p.getQuantidade() %></td>
                    <td><%= p.getPrecoVenda() %></td>
                    <td>
                        <a
href="ServletProdutoFC?acao=formAlterar&id=<%= p.getIdProduto() %>"
class="btn btn-primary btn-sm">Alterar</a>
                        <a href="ServletProdutoFC?acao=excluir&id=<%=
p.getIdProduto() %>" class="btn btn-danger btn-sm">Excluir</a>
                    </td>
                </tr>
                <%
                        }
                    }
                %>
            </tbody>
        </table>


        <!-- Link para o JS do Bootstrap -->
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bu
ndle.min.js"></script>
    </body>
</html>
```

Código do ProdutoDados.jsp:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<%@page import="cadastroee.model.Produto"%>

<!DOCTYPE html>

<html>

    <head>

        <meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">

        <title>Dados do Produto</title>

        <!-- Link para o CSS do Bootstrap via CDN -->

        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">

    </head>

    <body class="container">

        <h1 class="mt-4">Dados do Produto</h1>


        <%

            Produto produto = (Produto)
request.getAttribute("produto");

            String acao = (produto == null) ? "incluir" : "alterar";

            String id = (produto != null) ?
String.valueOf(produto.getIdProduto()) : "";

            String nome = (produto != null) ? produto.getNome() : "";

            String quantidade = (produto != null) ?
String.valueOf(produto.getQuantidade()) : "";

            String preco = (produto != null) ?
String.valueOf(produto.getPrecoVenda()) : "";

            String textoBotao = (produto == null) ? "Adicionar
Produto" : "Alterar Produto";

        %>


        <form action="ServletProdutoFC" method="POST" class="form mt-
3">

            <input type="hidden" name="acao" value="<%= acao %>">


            <% if (produto != null) { %>

                <input type="hidden" name="id" value="<%= id %>">
```

```
            <% } %>


            <div class="mb-3">
                <label class="form-label">Nome:</label>
                <input type="text" name="nome" value="<%= nome %>"
class="form-control">
            </div>


            <div class="mb-3">
                <label class="form-label">Quantidade:</label>
                <input type="text" name="quantidade" value="<%=
quantidade %>" class="form-control">
            </div>


            <div class="mb-3">
                <label class="form-label">Preco de Venda:</label>
                <input type="text" name="precoVenda" value="<%= preco
%>" class="form-control">
            </div>


            <button type="submit" class="btn btn-primary"><%=
textoBotao %></button>
        </form>

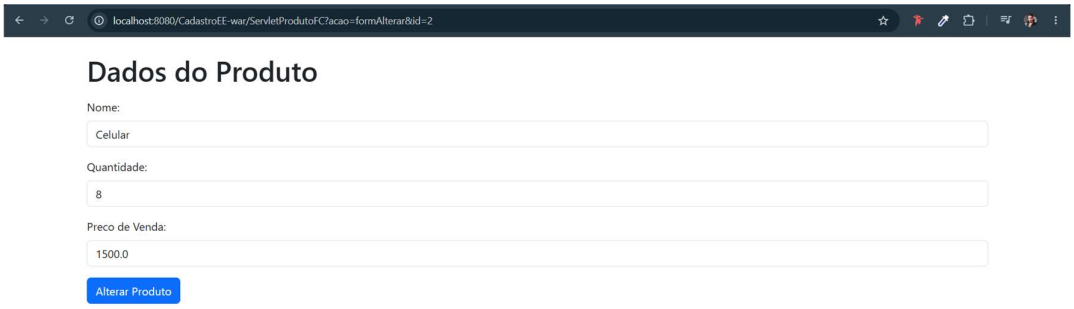

        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bu
ndle.min.js"></script>
    </body>
</html>
```

Resultado da Execução:

Listagem dos Produtos:



Modificar/Alterar Produto:



Adicionar Produto:



Análise e Conclusão

1º Procedimento (JPA, EJB e Servlets):

- Como é organizado um projeto corporativo no NetBeans?

Um projeto corporativo Java EE (Enterprise Application) é estruturado de forma modular utilizando um empacotamento EAR (Enterprise Archive). No NetBeans, ele é dividido em três subprojetos principais: um módulo EJB (-ejb) dedicado à camada de modelo, persistência de dados e regras de negócio, um módulo Web (-war) focado na camada de apresentação e controle (Servlets e JSPs), e

um projeto principal integrador que agrupa esses módulos para a implantação (deploy) no servidor.

- Qual o papel das tecnologias JPA e EJB na construção de um aplicativo para a plataforma Web no ambiente Java?

A JPA (Java Persistence API) atua como a ferramenta de mapeamento objeto relacional (ORM), sendo responsável por traduzir as classes Java (Entidades) para tabelas no banco de dados e gerenciar a persistência sem a necessidade de escrever queries SQL manualmente. Os EJBs (Enterprise JavaBeans) atuam na camada de negócio, processando as regras da aplicação e gerenciando transações e segurança de forma automática pelo contêiner do servidor de aplicações.

- Como o NetBeans viabiliza a melhoria de produtividade ao lidar com as tecnologias JPA e EJB?

O NetBeans oferece assistentes automatizados (Wizards) que geram códigos estruturais repetitivos. Por exemplo, ele permite criar automaticamente as "Classes de Entidade a partir do Banco de Dados" (JPA) e gerar as interfaces e classes Facade através da opção "Session Beans for Entity Classes" (EJB). Isso elimina a necessidade de digitação manual de mapeamentos e métodos CRUD básicos.

- O que são Servlets, e como o NetBeans oferece suporte à construção desse tipo de componentes em um projeto Web?

Servlets são classes Java executadas no servidor web que interceptam requisições HTTP do cliente, processam a lógica de controle e devolvem uma resposta. O NetBeans suporta sua criação através de assistentes (Novo > Servlet) que geram automaticamente a estrutura da classe, os métodos principais (doGet e doPost) e realizam a configuração de rotas via anotações (@WebServlet) ou atualização do arquivo web.xml.

- Como é feita a comunicação entre os Servlets e os Session Beans do pool de EJBs?

A comunicação ocorre por meio do mecanismo de injeção de dependências provido pelo contêiner Java EE. No Servlet, utiliza-se a anotação @EJB acima da declaração de um atributo tipado com a interface local do Session Bean

(FacadeLocal). Com isso, o servidor instancia e injeta o EJB automaticamente, permitindo que o Servlet invoque seus métodos de negócio diretamente.

2º Procedimento (Interface Cadastral):

- Como funciona o padrão Front Controller, e como ele é implementado em um aplicativo Web Java, na arquitetura MVC?

O padrão Front Controller define um ponto de entrada único e centralizado para todas as requisições da aplicação. Na arquitetura MVC em Java Web, ele é implementado por um Servlet central que captura todas as requisições, analisa um parâmetro de ação (ex: acao=listar), interage com o Modelo (EJBs) para executar a lógica necessária e, por fim, direciona o fluxo para a Visão (JSP) correspondente para renderizar a resposta.

- Quais as diferenças e semelhanças entre Servlets e JSPs?

Semelhanças: Ambos são tecnologias Java executadas no servidor web com o objetivo de processar requisições e gerar respostas HTTP dinâmicas para o cliente.

Diferenças: Os Servlets são arquivos de código-fonte Java puro focados na lógica de controle, tornando trabalhosa a escrita de código HTML em seu interior. As JSPs (JavaServer Pages) são arquivos baseados em marcação HTML que permitem a inserção de trechos de código Java de forma embutida, sendo ideais para a camada de visualização e apresentação gráfica.

- Qual a diferença entre um redirecionamento simples e o uso do método forward, a partir do RequestDispatcher? Para que servem parâmetros e atributos nos objetos HttpRequest?

O redirecionamento simples (sendRedirect) envia uma resposta ao navegador forçando-o a fazer uma requisição HTTP totalmente nova para outra URL, o que causa a perda dos dados da requisição anterior. Já o forward ocorre internamente no servidor: ele transfere o controle para outra página sem que o navegador saiba, mantendo a URL original e preservando os dados da requisição. Parâmetros são as informações enviadas pelo cliente (textos de formulários ou variáveis na URL). Atributos são objetos que o próprio servidor "guarda" dentro do objeto da requisição programaticamente para compartilhar dados entre um Servlet e uma página JSP durante aquele fluxo.

3º Procedimento (Melhoria de Design com Bootstrap):

- Como o framework Bootstrap é utilizado?

Ele é utilizado através da inclusão de links apontando para seus arquivos externos de CSS e JavaScript (geralmente via CDN ou baixados localmente) na seção <head> e ao final do <body> das páginas HTML/JSP. Uma vez importado, basta adicionar suas classes pré-definidas (como btn, table, container, etc.) diretamente nas tags HTML para aplicar a estilização visual instantaneamente.

- Por que o Bootstrap garante a independência estrutural do HTML?

O Bootstrap trabalha com classes CSS desacopladas da semântica. Isso significa que a estrutura do seu documento HTML permanece focada no significado do conteúdo (tabelas para dados tabulares, formulários para entradas), enquanto todo o comportamento visual e layout são controlados unicamente pelas classes aplicadas. Isso permite modificar drasticamente o design de uma página apenas trocando nomes de classes, sem a necessidade de reescrever a hierarquia de marcação do HTML.

- Qual a relação entre o Bootstrap e a responsividade da página?

O Bootstrap foi construído sob o conceito de *Mobile First*. Ele possui um poderoso sistema de grid fluido embutido e utiliza *media queries* nativas do CSS3 que detectam a resolução da tela do usuário. Com base nessas informações, ele reposiciona, redimensiona ou oculta elementos automaticamente, garantindo que a página se adapte e funcione de forma legível tanto em monitores grandes quanto em tablets ou smartphones.